02 - 블록체인 코어 시스템

개요

블록체인 코어 시스템은 Geth의 핵심 구성 요소로, 블록체인 상태 관리, 블록 처리, 트랜잭션 실행을 담당합니다. 이 시스템은 `core` 패키지에 구현되어 있으며, 주요 구조체인 `BlockChain`, `StateDB`, `StateProcessor` 를 통해 이더리움 프로토콜의 핵심 로직을 구현합니다.

전체 시스템에서 블록체인 코어는 P2P 네트워크로부터 받은 블록을 검증하고, EVM을 통해 트랜잭션을 실행하며, 최종적으로 상태를 데이터베이스에 저장하는 역할을 수행합니다.

핵심 구조체 및 인터페이스

BlockChain 구조체

`BlockChain` 은 전체 블록체인을 관리하는 핵심 구조체입니다:

```
type BlockChain struct {
chainConfig *params.ChainConfig // 체인 및 네트워크 설정
cfg *BlockChainConfig // 블록체인 설정

db ethdb.Database // 저수준 영구 데이터베이스
snaps *snapshot.Tree // 빠른 트라이 리프 접근을 위한 스냅샷 트리
triegc *prque.Prque[int64, common.Hash] // 트라이 가비지 컬렉션을 위한 우선순위 큐
triedb *triedb.Database // 트라이 노드 유지를 위한 데이터베이스 핸들러
statedb *state.CachingDB // 임포트 간 재사용을 위한 상태 데이터베이스
```

```
hc *HeaderChain // 헤더 체인
currentBlock atomic.Pointer[types.Header] // 현재 체인의 헤드
currentSnapBlock atomic.Pointer[types.Header] // 현재 스냅 동기화 헤드
engine consensus.Engine // 합의 엔진
validator Validator // 블록 및 상태 검증자 인터페이스
processor Processor // 블록 트랜잭션 처리자 인터페이스
}
```

StateDB 구조체

`StateDB` 는 이더리움 상태를 관리하는 핵심 구조체입니다:

```
type StateDB struct {
db Database // 데이터베이스 인터페이스
prefetcher *triePrefetcher // 트라이 프리페처
reader Reader // 상태 리더
trie Trie // 상태 트라이

originalRoot common.Hash // 변경 전 원본 루트

// 'live' 객체들을 보관하는 맵 (상태 전환 중 수정됨)
stateObjects  map[common.Address]*stateObject
```

```
// 삭제된 객체들을 보관하는 맵
stateObjectsDestruct  map[common.Address]*stateObject

// 트랜잭션 경계에서 발생한 계정 변경사항
mutations  map[common.Address]*mutation

refund  uint64  // 환불 카운터

// 트랜잭션 컨텍스트 및 로그
thash common.Hash
txIndex  int
logs  map[common.Hash][]*types.Log

// 접근 목록 (EIP-2930)
accessList *accessList
accessEvents *AccessEvents

// 임시 저장소 (EIP-1153)
transientStorage transientStorage

// 상태 수정 저널 (스냅샷 및 되돌리기의 백본)
journal *journal
```

```
}
```

StateProcessor 구조체

`StateProcessor` 는 트랜잭션 처리를 담당하는 구조체입니다:

```
type StateProcessor struct {
config *params.ChainConfig // 체인 설정 옵션
chain *HeaderChain // 정규 헤더 체인
}
```

주요 프로세스

1. 블록 삽입 과정 (InsertChain)

`BlockChain.InsertChain()` 메서드는 새로운 블록들을 체인에 삽입하는 핵심 함수입니다:

```
func (bc *BlockChain) InsertChain(chain types.Blocks) (int, error) {
// 삽입할 블록이 있는지 확인
if len(chain) == 0 {
return 0, nil
}
// 실제 삽입 로직 호출
_, n, err := bc.insertChain(chain, true, false)
return n, err
```

```
}
```

삽입 과정의 주요 단계:

1. **사전 검증**: 블록 헤더와 구조 검증
2. **상태 전환**: 각 트랜잭션 실행 및 상태 업데이트
3. **합의 검증**: 합의 엔진을 통한 블록 검증
4. **체인 업데이트**: 새로운 블록을 정규 체인에 추가
5. **이벤트 발생**: 체인 업데이트 이벤트 전파

2. 블록 조회 과정 (GetBlock)

`BlockChain.GetBlock()` 메서드는 해시와 번호로 블록을 조회합니다:

```
func (bc *BlockChain) GetBlock(hash common.Hash, number  uint64) *types.Block {
// 캐시에서 먼저 확인
if  block, ok := bc.blockCache.Get(hash);  ok {
return  block
}
// 데이터베이스에서 블록 조회
block := rawdb.ReadBlock(bc.db, hash, number)
if  block == nil {
return  nil
}
// 캐시에 저장 후 반환
bc.blockCache.Add(hash, block)
```

```
return  block
}
```

3. 트랜잭션 처리 과정 (StateProcessor.Process)

`StateProcessor.Process()` 메서드는 블록 내 모든 트랜잭션을 처리합니다:

```
func (p *StateProcessor) Process(block *types.Block, statedb *state.StateDB, cfg
var (
receipts types.Receipts
usedGas = new(uint64)
header = block.Header()
blockHash = block.Hash()
blockNumber = block.Number()
allLogs []*types.Log
gp = new(GasPool).AddGas(block.GasLimit())
)

// 하드포크 규칙 적용
if  p.config.DAOForkSupport && p.config.DAOForkBlock != nil &&
p.config.DAOForkBlock.Cmp(block.Number()) == 0 {
misc.ApplyDAOHardFork(statedb)
}
```

```go
// EVM 컨텍스트 생성
context := NewEVMBlockContext(header, p.chain, nil)
evm := vm.NewEVM(context, statedb, p.config, cfg)


// 각 트랜잭션 처리
for i, tx := range block.Transactions() {
// 트랜잭션을 메시지로 변환
msg, err := TransactionToMessage(tx, signer, header.BaseFee)
if err != nil {
return nil, fmt.Errorf("could not apply tx %d [%v]: %w", i, tx.Hash().Hex(), er
}
// 트랜잭션 컨텍스트 설정
statedb.SetTxContext(tx.Hash(), i)


// EVM으로 트랜잭션 실행
receipt, err := ApplyTransactionWithEVM(msg, gp, statedb, blockNumber, blockHash
if err != nil {
return nil, fmt.Errorf("could not apply tx %d [%v]: %w", i, tx.Hash().Hex(), er
}
receipts = append(receipts, receipt)
allLogs = append(allLogs, receipt.Logs...)
}
```

```
return &ProcessResult{
Receipts: receipts,
Logs: allLogs,
GasUsed: *usedGas,
}, nil
}
```

트랜잭션 처리 단계:

1. **가스 풀 초기화**: 블록 가스 한도로 가스 풀 생성
2. **하드포크 규칙 적용**: 특정 블록에서의 프로토콜 변경사항 적용
3. **EVM 컨텍스트 생성**: 블록 컨텍스트와 EVM 인스턴스 생성
4. **트랜잭션 반복 처리**: 각 트랜잭션을 순차적으로 실행
5. **결과 수집**: 영수증과 로그를 수집하여 반환

상태 관리 메커니즘

StateDB의 주요 메서드들

1. 잔액 관련 메서드

```
// GetBalance - 주소의 잔액 조회
func (s *StateDB) GetBalance(addr common.Address) *uint256.Int {
stateObject := s.getStateObject(addr)
if  stateObject != nil {
return  stateObject.Balance()
```

```
}
return common.U2560
}

// SetBalance - 주소의 잔액 설정
func (s *StateDB) SetBalance(addr common.Address, amount *uint256.Int, reason tr
stateObject := s.getOrNewStateObject(addr)
if stateObject != nil {
stateObject.SetBalance(amount)
}
}
```

2. 논스 관련 메서드

```
// GetNonce - 주소의 논스 조회
func (s *StateDB) GetNonce(addr common.Address) uint64 {
stateObject := s.getStateObject(addr)
if stateObject != nil {
return stateObject.Nonce()
}
return 0
}
```

```go
// SetNonce - 주소의 논스 설정
func (s *StateDB) SetNonce(addr common.Address, nonce uint64) {
stateObject := s.getOrNewStateObject(addr)
if stateObject != nil {
stateObject.SetNonce(nonce)
}
}
```

3. 코드 관련 메서드

```go
// GetCode - 주소의 컨트랙트 코드 조회
func (s *StateDB) GetCode(addr common.Address) []byte {
stateObject := s.getStateObject(addr)
if stateObject != nil {
return stateObject.Code(s.db)
}
return nil
}

// SetCode - 주소의 컨트랙트 코드 설정
func (s *StateDB) SetCode(addr common.Address, code []byte) {
stateObject := s.getOrNewStateObject(addr)
if stateObject != nil {
```

```
stateObject.SetCode(crypto.Keccak256Hash(code), code)
}
}
```

상태 객체 관리

StateDB는 상태 객체(`stateObject`)를 통해 개별 계정의 상태를 관리합니다:

```
// getStateObject - 상태 객체 조회 (존재하지 않으면 nil 반환)
func (s *StateDB) getStateObject(addr common.Address) *stateObject {
if obj := s.stateObjects[addr]; obj != nil {
return obj
}
// 데이터베이스에서 로드
data, err := s.reader.GetAccount(addr)
if err != nil {
s.setError(err)
return nil
}
if data == nil {
return nil
}
// 새 상태 객체 생성
obj := newObject(s, addr, data)
```

```
s.stateObjects[addr] = obj
return obj
}

// getOrNewStateObject - 상태 객체 조회 또는 생성
func (s *StateDB) getOrNewStateObject(addr common.Address) *stateObject {
stateObject := s.getStateObject(addr)
if stateObject == nil {
stateObject = s.createObject(addr)
}
return stateObject
}
```

코드 예시

1. 블록체인 초기화 및 블록 삽입

```
// 블록체인 생성
blockchain, err := core.NewBlockChain(db, genesis, engine, config)
if err != nil {
log.Fatal("Failed to create blockchain:", err)
}
```

```go
// 새로운 블록들을 체인에 삽입
blocks := []*types.Block{block1, block2, block3}
index, err := blockchain.InsertChain(blocks)
if  err != nil {
log.Printf("Failed to insert block at index %d: %v", index, err)
}

// 최신 블록 조회
latestBlock := blockchain.CurrentBlock()
fmt.Printf("Current block number: %d\n", latestBlock.Number.Uint64())
```

2. 상태 데이터베이스 사용

```go
// 상태 데이터베이스 생성
statedb, err := state.New(root, state.NewDatabase(db), nil)
if  err != nil {
log.Fatal("Failed to create state database:", err)
}

// 계정 잔액 조회
addr := common.HexToAddress("0x1234567890123456789012345678901234567890")
balance := statedb.GetBalance(addr)
fmt.Printf("Account balance: %s\n", balance.String())
```

```go
// 계정 잔액 설정
newBalance := uint256.NewInt(1000000000000000000) // 1 ETH in wei
statedb.SetBalance(addr, newBalance, tracing.BalanceChangeTransfer)

// 논스 조회 및 설정
nonce := statedb.GetNonce(addr)
statedb.SetNonce(addr, nonce+1)

// 상태 커밋
root, err := statedb.Commit(true)
if  err != nil {
log.Fatal("Failed to commit state:", err)
}
```

3. 트랜잭션 처리

```go
// StateProcessor 생성
processor := core.NewStateProcessor(chainConfig, headerChain)

// 블록 처리
result, err := processor.Process(block, statedb, vm.Config{})
if  err != nil {
```

```
log.Fatal("Failed to process block:", err)
}

// 처리 결과 확인
fmt.Printf("Gas used: %d\n", result.GasUsed)
fmt.Printf("Number of receipts: %d\n", len(result.Receipts))
fmt.Printf("Number of logs: %d\n", len(result.Logs))

// 각 영수증 확인
for  i, receipt := range  result.Receipts {
fmt.Printf("Transaction %d status: %d\n", i, receipt.Status)
fmt.Printf("Gas used: %d\n", receipt.GasUsed)
}
```

최적화 및 성능

1. 캐싱 메커니즘

BlockChain은 여러 레벨의 캐시를 사용하여 성능을 최적화합니다:

- **blockCache**: 최근 조회된 블록들을 메모리에 캐시
- **bodyCache**: 블록 바디를 별도로 캐시
- **receiptsCache**: 트랜잭션 영수증을 캐시
- **txLookupCache**: 트랜잭션 조회를 위한 캐시

2. 상태 프리페칭

StateDB는 트라이 프리페처를 사용하여 상태 접근을 최적화합니다:

```
// 프리페처를 통한 상태 미리 로드
if s.prefetcher != nil {
s.prefetcher.prefetch(s.originalRoot, [][]byte{addr.Bytes()})
}
```

3. 스냅샷 시스템

빠른 상태 접근을 위해 스냅샷 시스템을 활용합니다:

```
// 스냅샷에서 계정 데이터 조회
if s.snap != nil {
if acc, err := s.snap.Account(crypto.Keccak256Hash(addr.Bytes())); err == nil
return acc, nil
}
}
```

디버깅 및 모니터링

1. 메트릭 수집

블록체인 코어 시스템은 다양한 메트릭을 수집합니다:

```
// 블록 삽입 시간 측정
defer blockInsertTimer.UpdateSince(time.Now())
```

```
// 가스 처리량 측정
chainMgaspsMeter.Mark(int64(gasUsed))

// 상태 접근 시간 측정
defer  accountReadTimer.UpdateSince(time.Now())
```

2. 로깅

중요한 이벤트들은 구조화된 로그로 기록됩니다:

```
log.Info("Imported new chain segment",
"blocks", len(chain),
"txs", txs,
"mgas", float64(gas)/1000000,
"elapsed", common.PrettyDuration(time.Since(start)))
```

3. 이벤트 시스템

블록체인 상태 변화는 이벤트를 통해 다른 컴포넌트에 전파됩니다:

```
// 새로운 체인 헤드 이벤트 발생
bc.chainHeadFeed.Send(ChainHeadEvent{Block: block})

// 새로운 블록 이벤트 발생
```

```
bc.chainFeed.Send(ChainEvent{Block: block, Hash: block.Hash()})
```

관련 컴포넌트

의존성 관계

- **Storage Layer (07)**: 블록과 상태 데이터의 영구 저장
- **Consensus Engine (04)**: 블록 검증 및 합의 메커니즘
- **EVM Layer (05)**: 트랜잭션 실행 및 스마트 컨트랙트 처리
- **P2P Network (03)**: 새로운 블록 수신 및 전파

상호작용 방식

1. **P2P → BlockChain**: 새로운 블록 수신 시 `InsertChain()` 호출
2. **BlockChain → Consensus**: 블록 검증을 위해 합의 엔진 사용
3. **BlockChain → StateProcessor**: 트랜잭션 처리를 위해 `Process()` 호출
4. **StateProcessor → EVM**: 각 트랜잭션 실행을 위해 EVM 사용
5. **StateDB → Storage**: 상태 변경사항을 데이터베이스에 저장

이러한 구조를 통해 블록체인 코어 시스템은 이더리움 프로토콜의 핵심 기능을 안정적이고 효율적으로 구현합니다.